



Parallel Computing Pointer Jumping

T. Bosse, R. Domogalla, H. M. Bucker
Advanced Computing, FSU Jena

Parallel Random Access Machine

- Abkürzung : PRAM
- Alle p Prozessoren können gleichzeitig lesen von und schreiben in gemeinsamen Speicher
- Alle p Prozessoren sind eng synchronisiert
- Laufzeit = Anzahl Speicherzugriffe
- Konstante Kosten für Speicherzugriff

Unterschiedliche PRAM-Modelle

- **Concurrent Read (CR)**: mehrere Prozesse können gleichzeitig auf dieselbe Speicherzelle lesend zugreifen
- **Exclusive Read (ER)**: keine zwei Prozesse können gleichzeitig auf dieselbe Speicherzelle lesend zugreifen
- **Concurrent Write (CW)**: analog für Schreiben
- **Exclusive Write (EW)**: analog für Schreiben

Zugriffsstrategien für CW

- *Common*: alle Prozesse schreiben den gleichen Wert
- *Arbitrary*: gespeichert wird Wert eines zufällig ausgewählten Prozesses
- *Priority*: gespeichert wird Wert eines festgelegten Prozesses (kleinster Index, ...)
- *Combining*: gespeichert wird die Verknüpfung der Werte aller Prozesse (Summe, ...)

Abstrakter Daten Type „Liste“

$$L : [a_1, a_2, \dots, a_n]$$

- Länge: n
- Position von a_i in lineare Anordnung: i
- Menge von Operationen auf Listen:

$q \leftarrow L.\text{head}$ The position q is set to the first position on the list L .

$q \leftarrow L.\text{next}(p)$ The position q is set to the position following the position p on list L .

$e \leftarrow L.\text{item}(p)$ The element e is set to the element at position p in the list.

(und umgekehrte Zuweisungen)

$\perp = \text{NIL}$

Implementierung „Linked List“



- Reihenfolge von Elementen ist nicht durch Reihenfolge von Adressen bestimmt, sondern durch Verbindungen
- Speicheradresse eines Elements an einer Position kann nur durch Traversierung der Verbindungen erfolgen
- Paralleler Zugriff auf Elemente nur durch mehrfache Instanziierung der Liste möglich

List Ranking: Definition

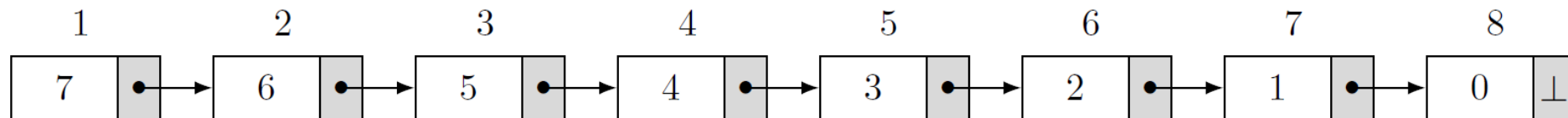


LIST RANKING

Input: A linked list L with n objects.

Output: For each object at position i in L , its distance from the end of the list which is recursively defined by

$$d(i) = \begin{cases} 0 & \text{if } L.\text{next}(i) = \text{NIL}, \\ d(L.\text{next}(i)) + 1 & \text{if } L.\text{next}(i) \neq \text{NIL}. \end{cases}$$



Annahme: Länge n unbekannt, sonst

$$d(i) = n - i \quad \text{for } i = 1, 2, \dots, n$$

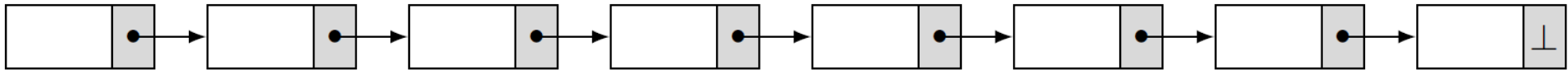
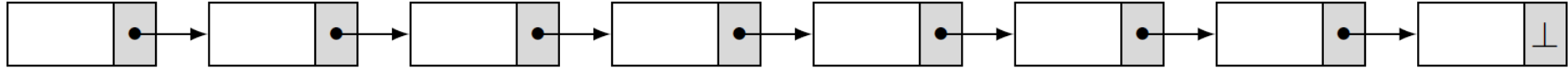
Serieller List Ranking Algorithmus

Algorithm 12.1: Given a nonempty list L , compute for each element $i = 1, 2, \dots, n$ the distance $d(i)$ to the end of that list.

```
1 Function  $d = \text{LISTRANKING}(L)$ 
2    $q \leftarrow L.\text{head}$ 
3    $k \leftarrow 1$ 
4   while  $L.\text{next}(q) \neq \text{NIL}$  do
5      $L.\text{item}(q) \leftarrow k$ 
6      $k \leftarrow k + 1$ 
7      $q \leftarrow L.\text{next}(q)$ 
8    $L.\text{item}(k) \leftarrow k$ 
9   for  $i = 1$  to  $k$  do
10     $d(i) \leftarrow k - L.\text{item}(i)$ 
```

Für Liste mit n Elementen: $\theta(n)$

Serieller Algorithmus am Beispiel



Quiz: Anzahl Speicherzugriffe?

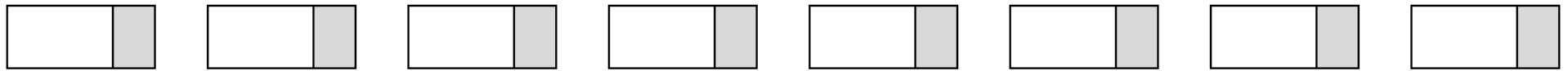
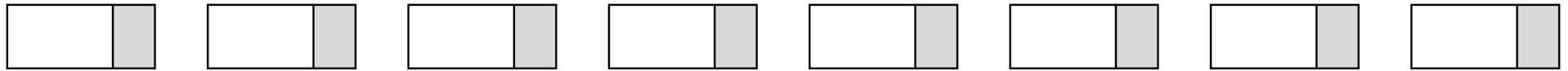
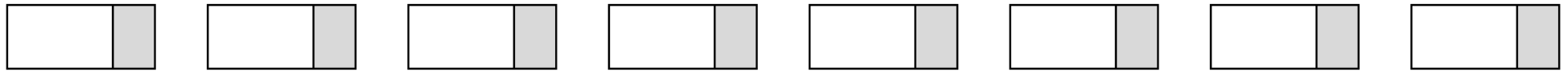
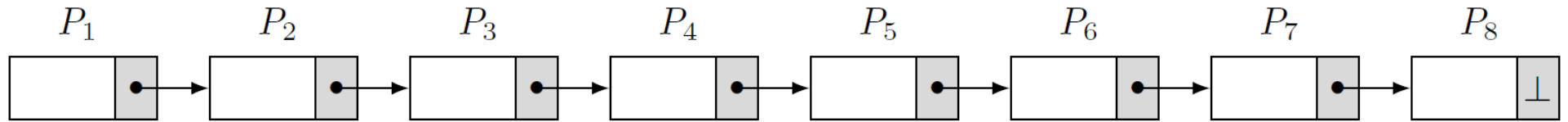
- a) $\theta(\log n)$
- b) $\theta(n)$
- c) $\theta(n \log n)$
- d) $\theta(n^2)$

Paralleler List Ranking Algorithmus

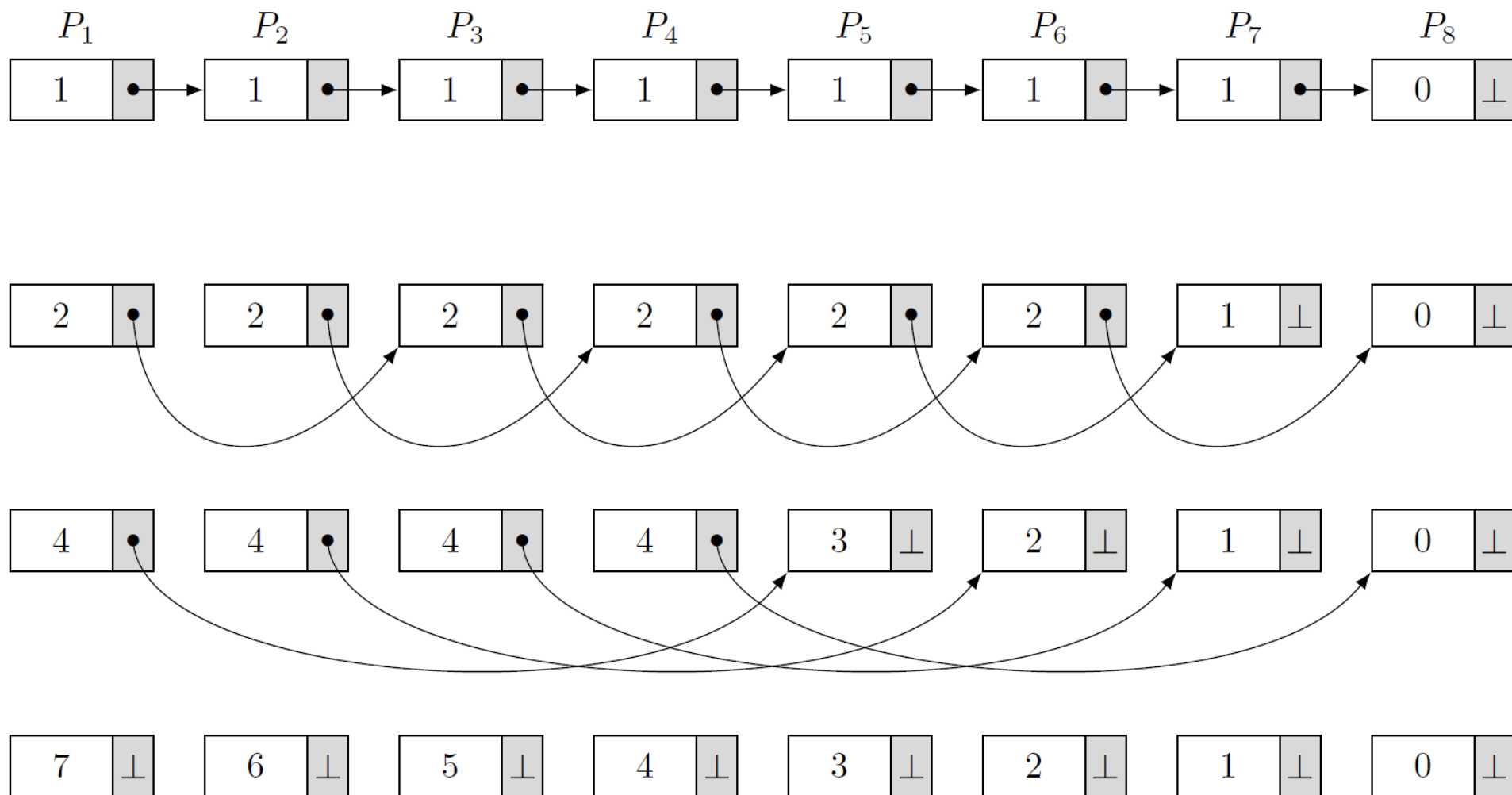
Algorithm 12.3: Given a nonempty list L with n elements, compute for each element $i = 1, 2, \dots, n$ the distance $d(i)$ to the end of that list using n processes.

```
1 Function  $d = \text{LISTRANKINGPARALLEL}(L)$ 
2   for each process  $i = 1$  to  $n$  do in parallel
3     if  $L.\text{next}(i) = \text{NIL}$  then
4        $L.\text{item}(i) \leftarrow 0$ 
5     else
6        $L.\text{item}(i) \leftarrow 1$ 
7   while there is an element  $i$  with  $L.\text{next}(i) \neq \text{NIL}$  do
8     for each process  $i = 1$  to  $n$  do in parallel
9       if  $L.\text{next}(i) \neq \text{NIL}$  then
10         $L.\text{item}(i) \leftarrow L.\text{item}(i) + L.\text{item}(L.\text{next}(i))$ 
11         $L.\text{next}(i) \leftarrow L.\text{next}(L.\text{next}(i))$ 
12   for each process  $i = 1$  to  $n$  do in parallel
13      $d(i) \leftarrow L.\text{item}(i)$ 
```

List Ranking: Beispiel



List Ranking: Beispiel

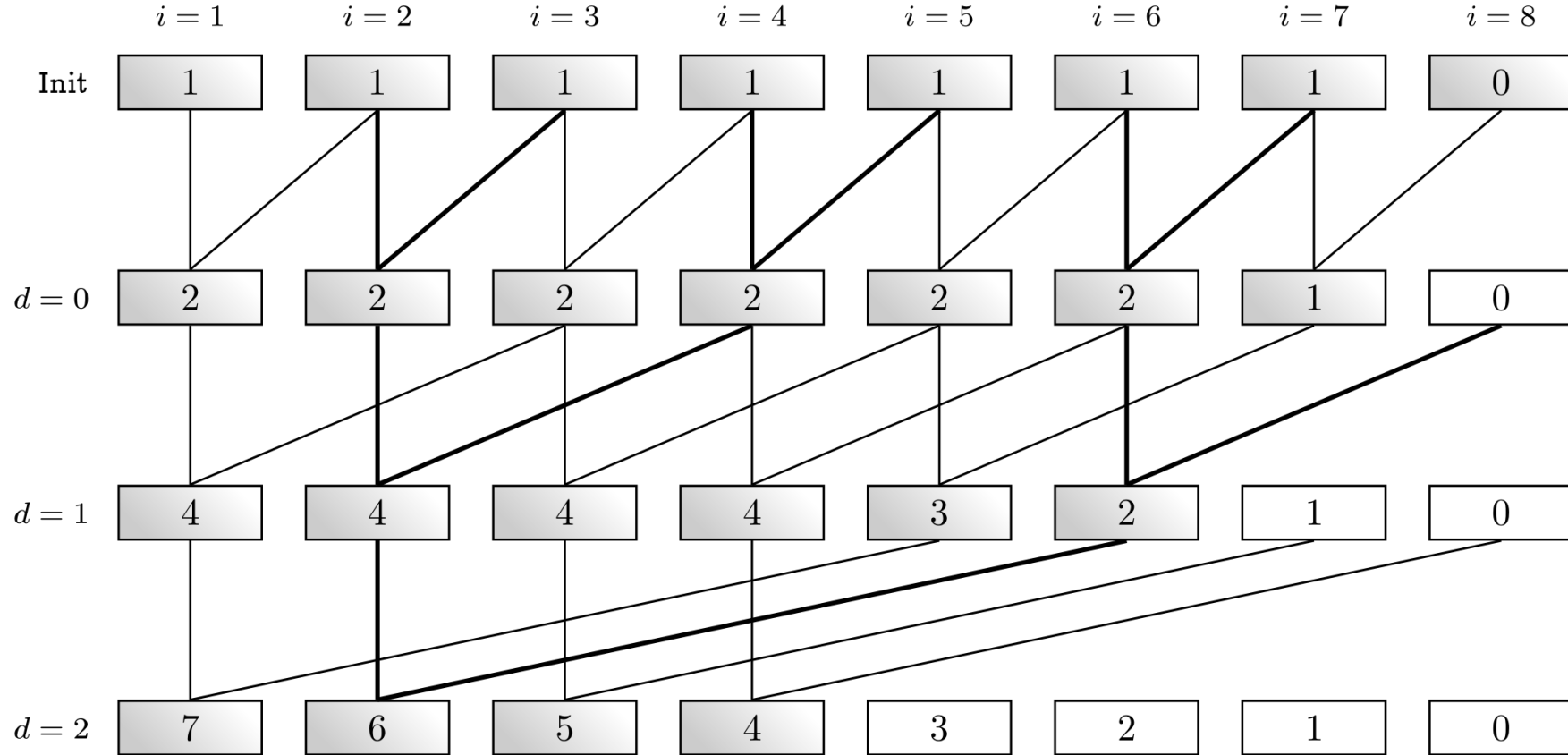


Quiz

Wie groß ist die Anzahl der Speicherzugriffe dieses Algorithmus für eine Liste mit n Elementen?

- a) $\theta(\log n)$
- b) $\theta(n)$
- c) $\theta(n \log n)$
- d) $\theta(n^2)$

List Ranking: Datenabhängigkeiten



Zusammenfassung

- ListRankingParallel Algorithmus ist nicht Arbeitseffizient
(*Algorithmus A ist „work-efficient“ bezgl. Algorithmus B, falls Arbeit von A bis auf konstanten Faktor gleich ist mit Arbeit von B.*)
- Dafür ist der Algorithmus aber Zeiteffizient.